



Data: 26/01/2026

Progetto: OikoNaos	Versione: 1.8
Documento: System Design	Data: 26/01/2026

Coordinatore del progetto:

Nome	Matricola
Luigi Potestà	0512120076

Partecipanti:

Nome	Matricola
Luigi Potestà	0512120076
Maria Antonietta Ruggiero	0512121126
Giulia Buonafine	0512119695
Mario Sinopoli	0512119113

Scritto da:	Giulia Buonafine
--------------------	------------------

Revision History

Data	Versione	Descrizione	Autore
15/11/2025	1.0	Prima stesura e definizione di Design Goals e Glossario	Mario Sinopoli
18/11/2025	1.1	Stesura delle sezioni Architettura corrente e Architettura proposta (Overview), inclusa introduzione dell'architettura 3-layer (Presentation, Application, Persistence)	Giulia Buonafine
20/11/2025	1.2	Prima bozza del documento SDD OikoNaos, con stesura iniziale di 'Gestione dati persistenti' e 'Controllo degli accessi e sicurezza'	Maria Antonietta Ruggiero
21/11/2025	1.3	Dettaglio della 'Decomposizione' in sottosistemi per i livelli View e Controller; integrazione dei componenti di 'Gestione Notifiche' e 'Gestione Risorse'	Luigi Potestà
22/11/2025	1.4	Ridefinizione sezioni 5.2 e 5.3 con aggiunta di diagrammi di decomposizione e mapping H/S; stesura delle sezioni 'Controllo flusso globale' e 'Boundary Conditions'	Giulia Buonafine

24/11/2025	1.5	Finalizzazione della sezione ‘Glossario dei Servizi dei Sottosistemi’ (Sez. 6), verificando la coerenza con i casi d'uso	Mario Sinopoli
26/11/2025	1.6	Miglioramento della formattazione	Luigi Potestà
05/12/2025	1.7	Aggiunta tabella degli accessi.	Giulia Buonafine Luigi Potestà
26/01/2026	1.8	Aggiornamento intero documento.	Luigi Potestà

Indice

1. Purpose	5
2. Audience	5
3.1 Purpose of the system.....	5
3.2. Design goals	6
3.3 Definitions, acronyms, abbreviations.....	7
3.4. References	8
3.5. Overview	8
4. Current Software Architecture	8
5. Proposed Software Architecture	10
5.1. Overview	10
5.2. Subsystem decomposition	10
5.3. Hardware/Software mapping.....	14
5.4 Persistent data management	15
5.5. Access control and security	17
5.6. Global software control.....	20
5.7. Boundary conditions	20
6. Subsystem Service Glossary.....	22

1. Purpose

Il presente documento di Requirements Analysis (RAD) descrive e formalizza i requisiti del progetto OikoNaos, una piattaforma web pensata per supportare la gestione quotidiana delle comunità di co-housing.

Il suo obiettivo principale è quello di raccogliere e organizzare in modo strutturato le esigenze espresse dal cliente, identificato nell'Azienda Coordinatrice, traducendole in requisiti funzionali e non funzionali chiari e verificabili.

Il RAD rappresenta un punto di riferimento condiviso tra cliente e gruppo di sviluppo, utile a garantire una corretta comprensione delle funzionalità previste e a consentire la tracciabilità dei requisiti lungo l'intero ciclo di vita del software. Il documento costituisce inoltre una guida per le successive fasi di progettazione, implementazione e verifica del sistema.

2. Audience

Il Requirement Analysis è rivolto a tutte le figure coinvolte nello sviluppo e nella validazione del sistema OikoNaos. In primo luogo, esso si indirizza al cliente, rappresentato dall'Azienda Coordinatrice, ente responsabile della gestione e del coordinamento delle comunità di co-housing e committente del progetto.

Il RAD è inoltre destinato al gruppo di sviluppo OikoNaos, che ricopre i ruoli di project management, analisi e progettazione del sistema, ed è incaricato di trasformare i requisiti raccolti in una soluzione software coerente con gli obiettivi del progetto.

Infine, il documento tiene conto delle esigenze degli utenti operativi della piattaforma, ossia i coinquilini e i supervisor di comunità. I supervisor sono figure che operano per conto dell'Azienda Coordinatrice e utilizzano la piattaforma per gestire e monitorare le attività della comunità, mentre i coinquilini rappresentano i principali destinatari dei servizi offerti dal sistema.

Introduction

3.1 Purpose of the system

Il sistema OikoNaos nasce con l'obiettivo di fornire una piattaforma digitale unica per la gestione delle comunità abitative di tipo co-housing, facilitando l'organizzazione delle attività quotidiane e la comunicazione tra i diversi attori coinvolti.

Attraverso OikoNaos, i coinquilini possono accedere ai principali servizi della comunità, tra cui la prenotazione di spazi e risorse comuni, la segnalazione di guasti e la gestione dei ticket di manutenzione, la partecipazione agli eventi comunitari e la consultazione della bacheca digitale, oltre al monitoraggio delle spese e dei contributi periodici.

Parallelamente, la piattaforma supporta i supervisor di comunità, figure operative che lavorano per conto dell'Azienda Coordinatrice, fornendo loro strumenti dedicati per la gestione delle prenotazioni, dei ticket, delle risorse condivise, degli eventi e delle spese. Tali funzionalità consentono ai supervisor

di amministrare in modo efficiente le comunità loro assegnate e di garantire il corretto funzionamento dei servizi offerti.

L'Azienda Coordinatrice assume quindi il ruolo di soggetto responsabile della gestione complessiva delle comunità, mentre la piattaforma OikoNaos rappresenta lo strumento attraverso cui supervisor e coinquilini interagiscono operativamente.

In questo contesto, OikoNaos si propone come un supporto concreto alla collaborazione e alla comunicazione all'interno delle comunità di co-housing, contribuendo a migliorare l'esperienza abitativa attraverso l'uso della tecnologia.

3.2. Design goals

L'architettura di OikoNaos è guidata da un insieme di *design goals* derivati dai requisiti funzionali e non funzionali definiti nel Requirements Analysis Document (RAD). Tali obiettivi hanno orientato le scelte architetture e tecnologiche, garantendo coerenza tra requisiti, progettazione e implementazione effettiva del sistema.

DG01 – Modularità

Il sistema è progettato secondo una struttura modulare, basata sull'architettura **Model–View–Controller (MVC)**, che consente di separare chiaramente la presentazione, la logica applicativa e la gestione dei dati. La suddivisione in sottosistemi funzionali (autenticazione, gestione prenotazioni, ticket, eventi, risorse e pagamenti) favorisce alta coesione interna, ridotto accoppiamento e facilita la manutenzione e l'evoluzione del sistema. Questo obiettivo è coerente con i requisiti di configurabilità ed estendibilità definiti nel RAD.

DG02 – Scalabilità e Performance

L'architettura è pensata per supportare un numero crescente di utenti e operazioni concorrenti, mantenendo tempi di risposta adeguati. L'adozione di un application server servlet-based, il modello *thread-per-request* e l'uso di un DBMS relazionale consentono al sistema di gestire carichi multiutente in modo stabile. La scalabilità è prevista principalmente a livello infrastrutturale (es. separazione dei nodi o moltiplicazione delle istanze), senza richiedere modifiche alla struttura logica dell'applicazione.

DG03 – Sicurezza dei dati

La sicurezza dei dati rappresenta un obiettivo centrale dell'architettura. Il sistema implementa meccanismi di autenticazione sicura, controllo degli accessi basato sui ruoli (RBAC) e protezione delle credenziali tramite hashing delle password. L'accesso alle funzionalità applicative è regolato lato server, prevenendo utilizzi non autorizzati. Gli aspetti relativi alla protezione delle comunicazioni e alla conformità normativa (es. GDPR) sono considerati a livello di requisiti e di deployment, mentre l'implementazione applicativa si concentra sulla protezione dei dati e delle operazioni sensibili.

DG04 – Affidabilità e Robustezza

Il sistema è progettato per mantenere un comportamento corretto e coerente anche in presenza di input non validi o condizioni di errore. I controller applicativi verificano le precondizioni delle operazioni e impediscono la creazione di stati incoerenti nel dominio applicativo. L'affidabilità è ulteriormente

supportata dall'uso di vincoli di integrità nel database e dalla gestione controllata delle eccezioni a livello applicativo.

DG05 – Supporto alla gestione multilivello

OikoNaos supporta la distinzione tra diversi livelli operativi attraverso un modello di accesso basato sui ruoli. L'architettura consente a **coinquilini** e **supervisori** di accedere a funzionalità differenti, in accordo con le rispettive responsabilità. Le operazioni amministrative di livello superiore, non destinate all'uso quotidiano tramite WebApp, sono considerate esterne al sistema applicativo. Questa scelta riduce la complessità dell'interfaccia e rafforza la sicurezza delle funzionalità critiche.

DG06 – Usabilità

La piattaforma è progettata per essere semplice e intuitiva, anche per utenti con competenze informatiche limitate. L'interfaccia web è organizzata in modo chiaro e coerente, favorendo una navigazione immediata e riducendo la possibilità di errore. L'obiettivo è supportare le attività quotidiane delle comunità di co-housing in modo rapido ed efficace, migliorando l'esperienza complessiva dell'utente.

DG07 – Portabilità

Il sistema utilizza tecnologie standard e ampiamente supportate (web application Java/Jakarta, application server Tomcat 10.x e DBMS MySQL), garantendo compatibilità con i principali browser moderni e ambienti di esecuzione. Questa scelta consente il deploy dell'applicazione su piattaforme differenti senza modifiche alla struttura del sistema, in linea con i requisiti di portabilità e compatibilità definiti nel RAD.

Nel loro insieme, questi design goals definiscono un'architettura coerente, robusta e orientata alla manutenibilità, assicurando l'allineamento tra requisiti, progettazione e implementazione concreta del sistema OikoNaos.

3.3 Definitions, acronyms, abbreviations

Nel contesto del presente documento si adottano le seguenti definizioni:

Co-housing	modello abitativo in cui più persone condividono spazi e servizi comuni, mantenendo al contempo alloggi privati autonomi.
Azienda Coordinatrice	ente responsabile della gestione complessiva della struttura residenziale, del coordinamento delle comunità e della supervisione delle attività amministrative e manutentive.
Supervisore	figura operativa che lavora per conto dell'Azienda Coordinatrice e utilizza la piattaforma OikoNaos per gestire e monitorare le attività della comunità.
Coinquilino	utente finale della piattaforma, abilitato all'utilizzo dei servizi di prenotazione, segnalazione, partecipazione agli eventi e consultazione delle spese.

Ticket	richiesta di intervento o segnalazione di un problema generata da un utente e tracciata dal sistema nel tempo.
Bacheca Eventi	sezione informativa della piattaforma dedicata alla pubblicazione e consultazione di eventi, comunicazioni e annunci relativi alla vita comunitaria
Risorsa condivisa	spazio o bene comune della comunità (ad esempio sale, attrezzature o ambienti comuni) soggetto a prenotazione e gestione tramite il sistema.
Tassa	contributo economico periodico assegnato ai coinquilini per la copertura delle spese comuni della comunità.
Penale	importo economico applicato a un coinquilino in seguito al mancato rispetto di regole o scadenze previste.

3.4. References

- **Requisiti funzionali:** Sezione 4.2 del RAD
- **Requisiti non funzionali:** Sezione 4.3 del RAD

3.5. Overview

Questa sezione introduttiva fornisce una visione generale del progetto OikoNaos e ne chiarisce finalità, struttura e terminologia adottata. Le sezioni successive approfondiscono in modo progressivo gli aspetti specifici dell'analisi dei requisiti.

In particolare, la Sezione 2 descrive lo stato attuale dei processi di gestione delle comunità di co-housing, mettendo in evidenza le principali criticità e le limitazioni legate all'utilizzo di strumenti non integrati o a procedure prevalentemente manuali.

La Sezione 3 presenta il sistema proposto, illustrandone le funzionalità previste, i requisiti funzionali e non funzionali e i modelli utilizzati per descriverne il comportamento e la struttura.

Infine, la Sezione 5 raccoglie il glossario dei termini principali, con l'obiettivo di garantire uniformità di linguaggio e chiarezza nella comunicazione tra tutte le parti coinvolte nel progetto.

4. Current Software Architecture

Attualmente, la gestione delle comunità di co-housing avviene in modo frammentato e con un livello di digitalizzazione limitato. Le attività quotidiane, come la prenotazione degli spazi comuni, la segnalazione di guasti o la diffusione di informazioni tra residenti e amministrazione, vengono spesso svolte tramite strumenti eterogenei, quali comunicazioni via e-mail, messaggi informali o moduli cartacei.

Questo approccio, sebbene possa risultare sufficiente in contesti di dimensioni ridotte, diventa rapidamente inefficiente in comunità più strutturate, dove il numero di utenti, risorse e attività da gestire cresce in modo significativo.

L'assenza di un sistema centralizzato genera numerose criticità operative. La gestione manuale delle prenotazioni degli spazi comuni comporta frequenti sovrapposizioni e disguidi, poiché non esistono meccanismi automatici in grado di controllare la disponibilità delle risorse o di prevenire richieste concorrenti. Analogamente, la segnalazione di guasti e richieste di manutenzione avviene in maniera informale, rendendo complesso il tracciamento delle segnalazioni e la gestione delle priorità di intervento.

Dal punto di vista amministrativo, la gestione delle spese comuni, delle tasse periodiche e delle eventuali penali è spesso affidata a registrazioni manuali o a fogli di calcolo non condivisi, con un conseguente aumento del rischio di errori e una ridotta trasparenza nei confronti dei residenti.

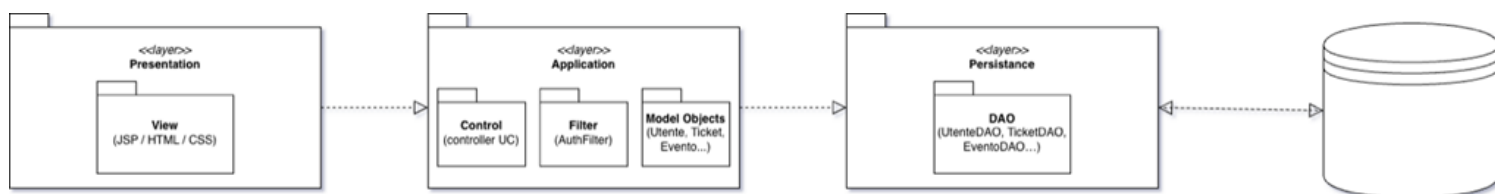
Anche la comunicazione interna risulta frammentata: l'assenza di una bacheca digitale o di un canale ufficiale rende difficoltosa la diffusione uniforme di annunci, eventi e comunicazioni importanti, incidendo negativamente sulla partecipazione alla vita comunitaria.

5. Proposed Software Architecture

5.1. Overview

L'architettura proposta per OikoNaos è una web application a tre livelli, progettata per centralizzare la gestione delle attività di co-housing (utenze, prenotazioni, ticket, eventi e risorse condivise) garantendo separazione delle responsabilità, manutenibilità e possibilità di evoluzione.

- **Presentazione (client):** interfaccia web fruibile tramite browser, realizzata tramite pagine e risorse web (View) e pensata per l'uso operativo da parte di **Coinquilini** e **Supervisor**. Le funzioni strettamente amministrative dell'Azienda Coordinatrice non sono esposte come area applicativa dedicata nella WebApp e, ove necessario, sono gestite tramite procedure esterne alla piattaforma.
- **Applicazione (server):** livello eseguito su application server (Tomcat) che implementa i casi d'uso tramite **Controller** (servlet) e meccanismi di controllo accessi (filter/sessioni). La logica applicativa coordina l'accesso ai dati attraverso componenti **DAO** e gestisce validazioni e regole operative (es. disponibilità risorse, stati ticket, operazioni del supervisore).
- **Persistenza (database):** livello responsabile della memorizzazione dei dati su DBMS MySQL, secondo lo schema relazionale definito dal progetto; l'accesso avviene tramite DAO e JDBC, mantenendo separata la logica di persistenza dal resto dell'applicazione.



5.2. Subsystem decomposition

La decomposizione in sottosistemi di OikoNaos segue l'architettura **Model–View–Controller (MVC)** tipica delle web application Servlet/JSP. La separazione tra presentazione, logica applicativa e gestione dei dati è ottenuta tramite **JSP** per la View, **Servlet/Filter** per il Controller e **Model + DAO (JDBC)** per l'accesso al database. I sottosistemi sono progettati per mantenere alta coesione e basso accoppiamento, in linea con i Design Goals. Il sistema è suddiviso nei seguenti sottosistemi, organizzati secondo i tre livelli della sua architettura: View, Controller e Model.

Nella View (Interfaccia Utente)

- **GUI Coinquilino (Area utente):** interfaccia web destinata ai residenti. Permette di autenticarsi/registrarsi, effettuare prenotazioni, creare e consultare ticket (con dettagli e allegati),

visualizzare la bacheca eventi e gestire iscrizioni, consultare spese/pagamenti e visualizzare le ricevute, oltre alle pagine di recupero/reset password.

- **GUI Supervisore (Area supervisore):** interfaccia web dedicata ai supervisori, accessibile tramite pagine JSP in cartella dedicata. Consente di gestire ticket e relativo storico aggiornamenti, gestire/monitorare prenotazioni, gestire risorse/richieste risorsa e monitorare la situazione tasse/pagamenti della comunità, oltre alla gestione degli eventi lato supervisore.
- **GUI Azienda Coordinatrice: non è presente come area operativa nella WebApp** nello stato attuale del codice. Le operazioni di tipo “centrale” (es. gestione account supervisore/operazioni amministrative) risultano demandate a procedure esterne alla piattaforma (es. gestione diretta su DB), e non a pagine/controller dedicati.

La View raccoglie input dagli utenti e presenta i risultati, demandando la logica applicativa ai controller.

Nella componente Controller (Logica dei casi d’uso)

Nel codice la logica applicativa è implementata principalmente in **Servlet Controller** e **Filter**, che orchestrano i casi d’uso e invocano i DAO per la persistenza.

- **Gestione Autenticazione e Sessione**
 - Login/Logout e gestione sessione utente.
 - Registrazione e validazione del codice identificativo (prima registrazione).
 - Controllo accessi tramite **Filter** (blocco utenti non autenticati e restrizione area supervisore).
- **Gestione Profilo e Credenziali**
 - Visualizzazione/aggiornamento dati utente e modifica credenziali.
- **Gestione Recupero Password**
 - Richiesta reset, generazione/validazione token, impostazione nuova password (workflow completo con token).
- **Gestione Prenotazioni**
 - Creazione e visualizzazione prenotazioni; lato supervisore consultazione/gestione operativa delle prenotazioni.
- **Gestione Ticket di Manutenzione e Allegati**
 - Creazione ticket, consultazione dettagli, aggiornamenti/stato lato supervisore.
 - Gestione file allegati: upload su filesystem e download controllato.
- **Gestione Eventi e Iscrizioni**
 - Visualizzazione bacheca eventi.
 - Iscrizione/disiscrizione e conferme lato utente; gestione eventi lato supervisore.
- **Gestione Risorse Condivise**
 - Gestione risorse e richieste risorsa (lato supervisore e/o lato utente in base alle pagine/controller presenti).
- **Gestione Pagamenti, Spese, Tasse e Ricevute**
 - Visualizzazione spese, conferma/registrazione pagamenti secondo i flussi presenti.
 - Generazione/visualizzazione ricevute.
 - Monitoraggio tasse/pagamenti lato supervisore.

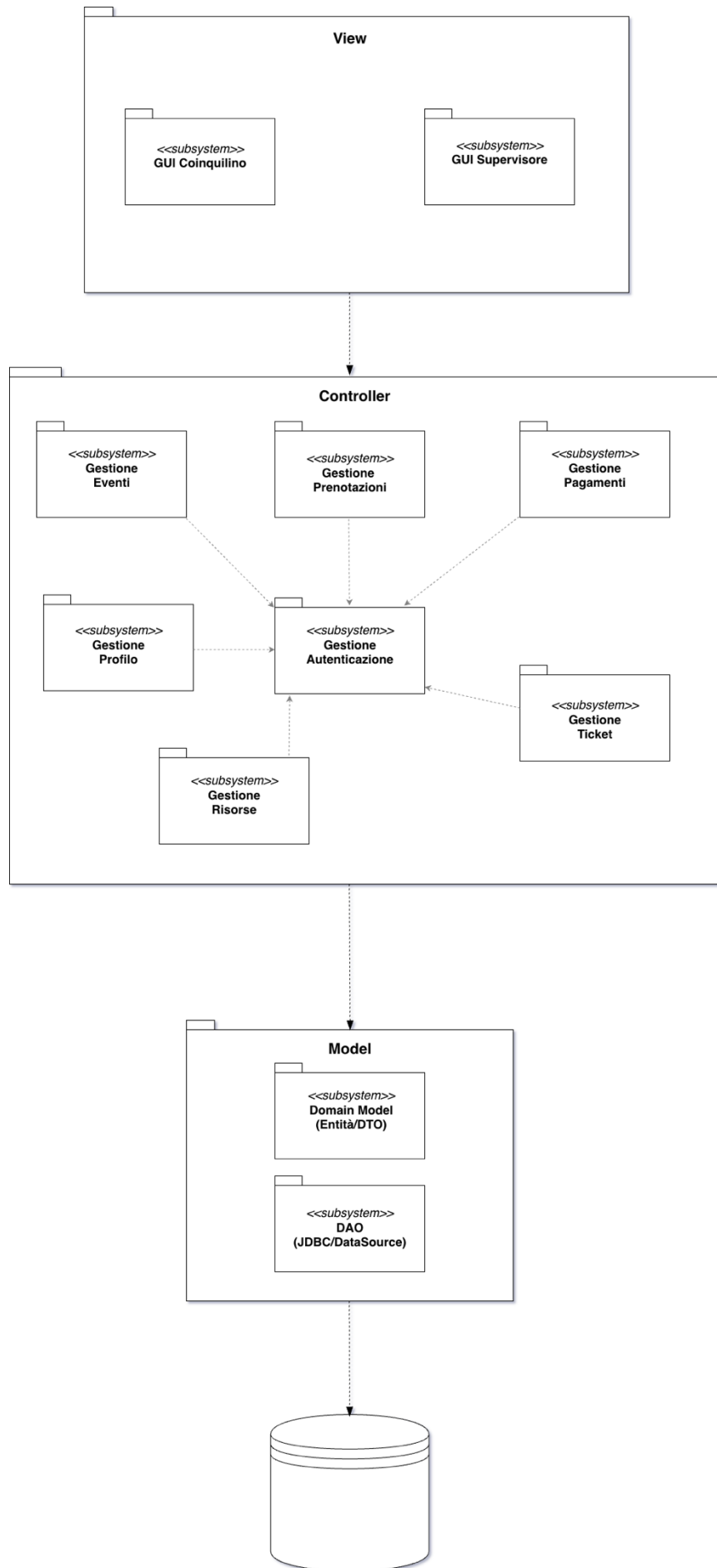
Ciascun sottosistema del Controller realizza uno o più casi d’uso del RAD, mantenendo la tracciabilità

tra requisiti e componenti implementativi.

Nella componente Model (Dati e Persistenza)

- **Entità del dominio (Model):** il progetto include oggetti dominio per rappresentare i concetti principali gestiti dalla piattaforma, tra cui (in base alle classi presenti): Utente, Credenziali, CodiceIdentificativo, Prenotazione, Ticket, AggiornamentoTicket, Evento, Pagamento, Ricevuta, TassaTrimestrale, TokenResetPassword, Risorsa, RichiestaRisorsa.
- **DTO / Strutture di trasferimento dati:** per alcuni dati (es. allegati) è presente un approccio DTO (es. **AllegatoDTO (path/metadati)**) usato dai controller/DAO per trasferire informazioni senza introdurre necessariamente un'entità dominio completa.
- **Persistenza (DAO + JDBC):** l'accesso al database è incapsulato in componenti **DAO** dedicati, che implementano operazioni CRUD e query applicative. Nel codice risultano presenti, tra gli altri: UserDAO, CredenzialiDAO, CodiceIdentificativoDAO, PrenotazioneDAO, TicketDAO, AggiornamentoTicketDAO, EventoDAO, PagamentoDAO, RicevutaDAO, TassaDAO, TokenResetPasswordDAO, RisorsaDAO, RichiestaRisorsaDAO, AllegatoDAO.

La comunicazione con il DB avviene tramite JDBC e DataSource, mantenendo separata la logica di persistenza dalla gestione dei casi d'uso.



5.3. Hardware/Software mapping

L'architettura di OikoNaos è una web application multilivello che separa chiaramente presentazione, logica applicativa e persistenza. Questa organizzazione favorisce modularità e manutenibilità e consente di far evolvere l'infrastruttura nel tempo senza modificare la struttura logica del sistema.

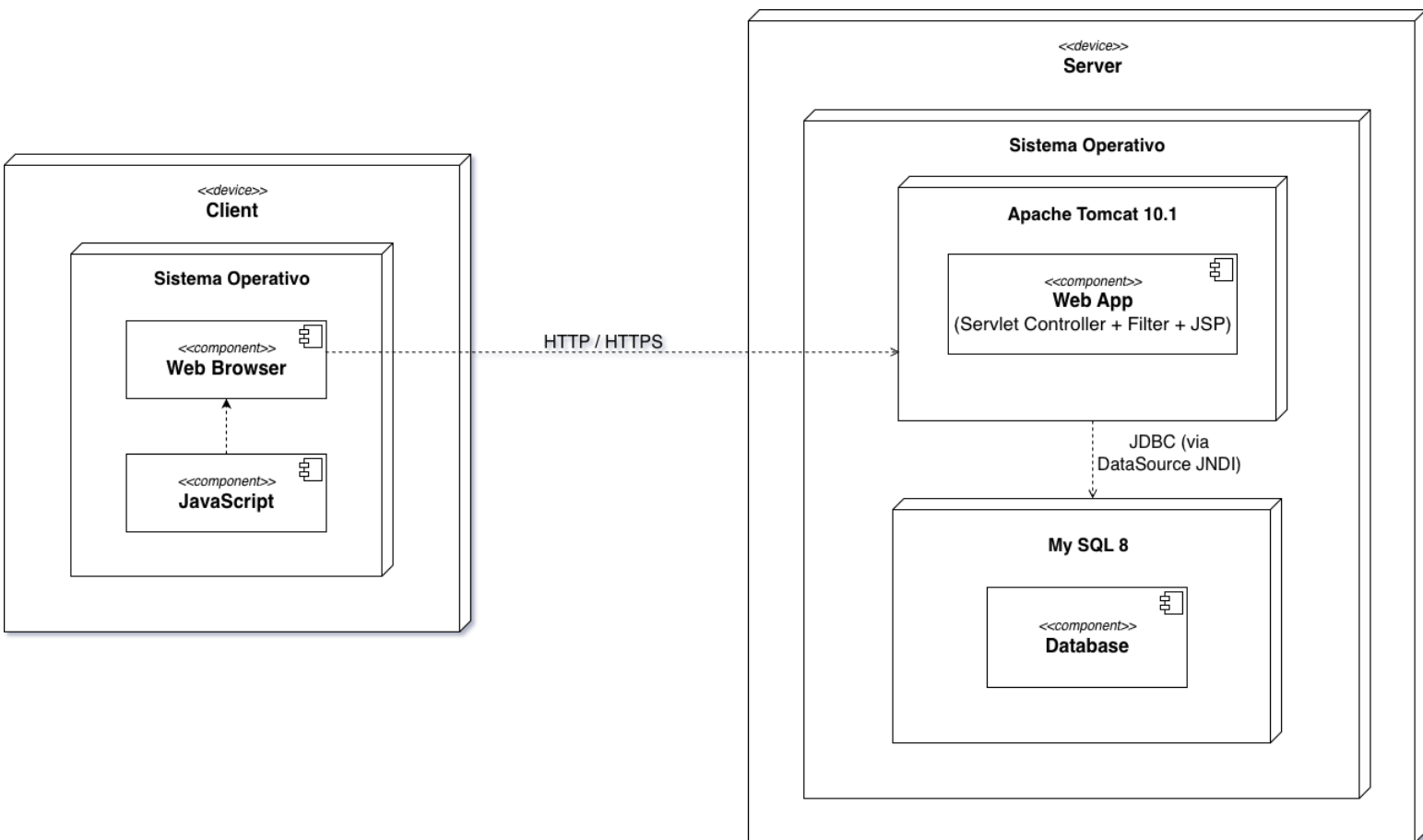
Il **nodo Client** è costituito da qualsiasi dispositivo dotato di un browser moderno (desktop o mobile). Sul client risiedono esclusivamente le componenti di presentazione: pagine JSP/HTML, fogli di stile e risorse statiche che vengono renderizzate nel browser. Il client non esegue logica di business: invia richieste HTTP verso il server e visualizza i risultati restituiti dall'applicazione.

Il **nodo Server** ospita l'applicazione Java eseguita su Apache Tomcat (nel progetto sono utilizzate API Jakarta compatibili con lo stack Tomcat 10.x). La logica applicativa è implementata principalmente tramite Servlet Controller (che gestiscono i casi d'uso) e Filter (per controllo accessi e sessione). Nel codice non è presente un layer "Services" separato: l'orchestrazione dei flussi avviene nei controller, che delegano le operazioni di lettura/scrittura dati ai componenti di persistenza.

La **persistenza** è realizzata tramite un DBMS MySQL e accesso ai dati con pattern DAO via JDBC. Le connessioni al database non sono aperte "a mano" direttamente nei controller: l'applicazione recupera una DataSource tramite JNDI (java:comp/env/jdbc/OikoNaosDB), quindi la gestione del pool di connessioni è demandata al container (Tomcat). Questo approccio migliora efficienza e robustezza rispetto alla creazione diretta di connessioni.

Per la **sicurezza**, alcune misure sono effettivamente implementate nel codice: in particolare l'hashing delle password tramite BCrypt e il controllo degli accessi tramite sessione e filtri/ruoli lato server. Al contrario, l'uso di HTTPS/TLS non è una caratteristica "applicativa" definita nel codice: è una scelta di deployment/configurazione del server (Tomcat/reverse proxy). Di conseguenza, nel System Design va trattato come requisito/decisione infrastrutturale, non come comportamento garantito dall'applicazione stessa.

Dal punto di vista dell'infrastruttura fisica, per semplicità è possibile collocare **application server e DBMS** sullo stesso nodo (scenario tipico di sviluppo o piccoli carichi), riducendo complessità e latenza. L'architettura resta comunque distribuibile: in caso di crescita del carico, è possibile separare il database su un nodo dedicato e mantenere una o più istanze Tomcat, eventualmente dietro bilanciamento, senza cambiare l'organizzazione logica MVC + DAO.



5.4 Persistent data management

La gestione dei dati persistenti in OikoNaos è affidata al sottosistema Model, che incapsula completamente l'accesso al database e separa la logica di persistenza dalla logica applicativa. Il sistema utilizza un database relazionale MySQL 8, progettato in coerenza con il dominio concettuale definito nel RAD e popolato con le principali informazioni gestite dalla piattaforma. Le entità persistenti rappresentano i concetti fondamentali del sistema, tra cui Utente, Credenziali, Codice Identificativo, Prenotazione, Ticket, Aggiornamento Ticket, Evento, Pagamento, Ricevuta, Tassa Trimestrale, Risorsa, Richiesta Risorsa e Token di Reset Password. I ruoli applicativi (es. Supervisore) sono modellati come attributi dell'utente e non come entità separate.

L'accesso ai dati è realizzato tramite componenti DAO (Data Access Object), ciascuno responsabile della gestione di una specifica entità o insieme di entità correlate. I DAO implementano le operazioni CRUD e le query applicative, fungendo da unico punto di contatto tra la logica applicativa e il DBMS. La comunicazione con il database avviene tramite JDBC, utilizzando una DataSource configurata via JNDI, demandando al container la gestione del pool di connessioni.

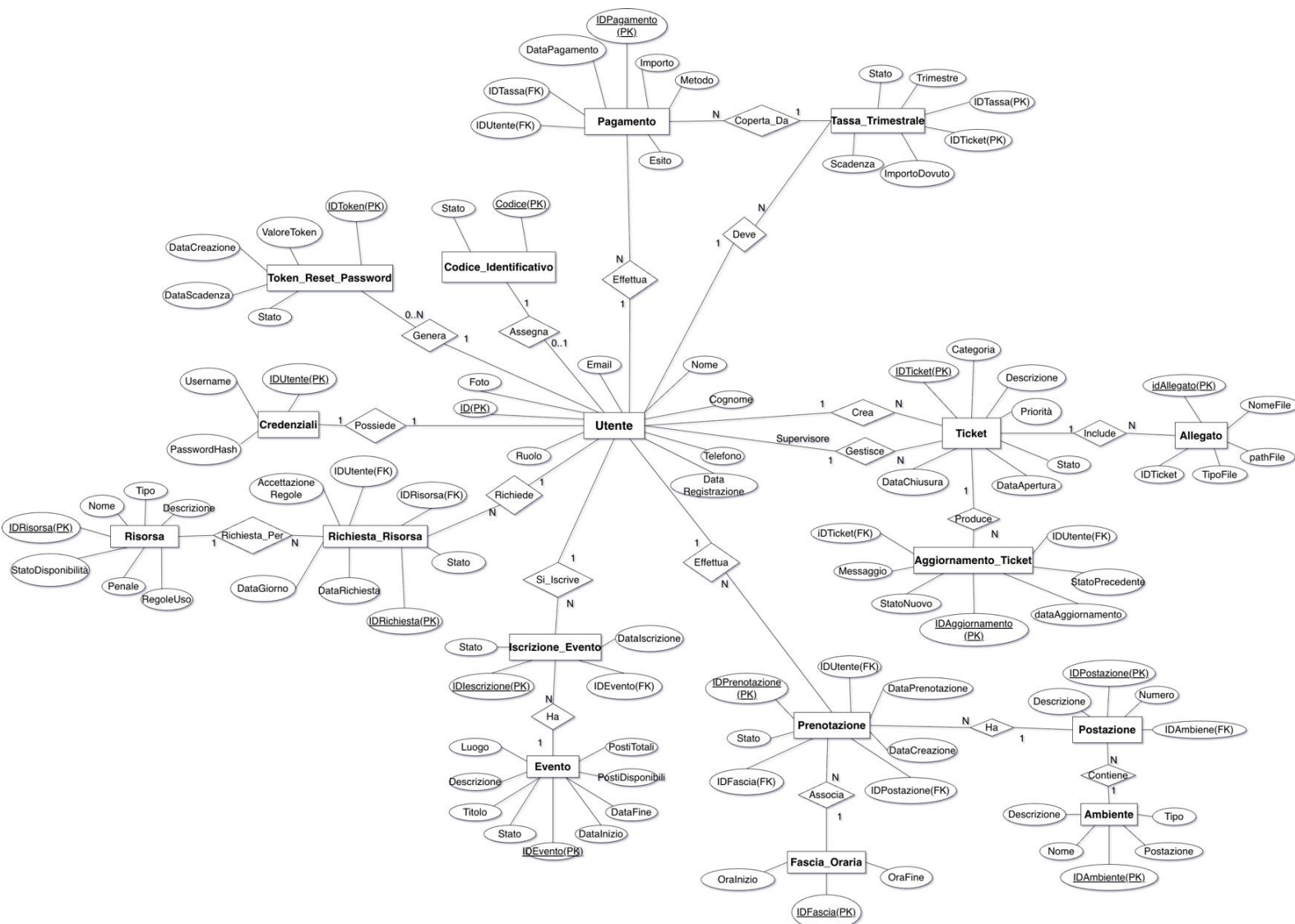
Le operazioni di persistenza sono eseguite in modalità autocommit, garantendo l'atomicità delle singole operazioni di lettura e scrittura. I vincoli di integrità referenziale sono assicurati a livello di database mediante chiavi primarie ed esterne, mentre ulteriori controlli applicativi, implementati nei controller,

prevedono la creazione di stati incoerenti (ad esempio prenotazioni non valide o operazioni non autorizzate).

I file multimediali associati ai ticket (allegati) non sono memorizzati direttamente nel database: essi vengono salvati nel file system del server, mentre nel database è persistito esclusivamente il percorso di riferimento e i metadati necessari al recupero controllato del file.

La sicurezza dei dati persistenti è garantita da più livelli di protezione. Le password degli utenti sono memorizzate esclusivamente in forma hashata tramite algoritmo BCrypt. Le query verso il database utilizzano PreparedStatement, riducendo il rischio di SQL Injection. L'accesso ai dati è ulteriormente regolato dai controlli di sessione e di ruolo implementati lato server, in coerenza con il modello RBAC effettivamente supportato dall'applicazione.

Le politiche di backup e ripristino del database sono considerate una responsabilità dell'infrastruttura di deployment e non sono implementate direttamente a livello applicativo; esse possono essere configurate in base all'ambiente di esercizio per garantire la continuità operativa del sistema.



5.5. Access control and security

L'accesso al sistema OikoNaos è regolato da un insieme coordinato di meccanismi di autenticazione, gestione delle sessioni e controllo degli accessi basato sui ruoli (RBAC), implementati lato server. L'obiettivo è garantire che ogni utente possa accedere esclusivamente alle funzionalità coerenti con il proprio ruolo e con lo stato del dominio applicativo.

Autenticazione e gestione delle sessioni

L'autenticazione degli utenti avviene tramite **credenziali verificate dal server**. Le password non sono mai memorizzate in chiaro, ma esclusivamente in forma **hashata** mediante l'algoritmo **BCrypt**, garantendo protezione contro accessi non autorizzati e compromissione delle credenziali.

A seguito dell'autenticazione, il sistema crea una **sessione HTTP** associata all'utente, utilizzata per identificare le richieste successive. La sessione viene invalidata in caso di logout o accesso non autorizzato, prevenendo utilizzi impropri o riutilizzo di sessioni non valide.

L'accesso alle risorse dell'applicazione è ulteriormente protetto da **filter lato server**, che intercettano le richieste HTTP e verificano: la presenza di una sessione valida e la corrispondenza tra il ruolo dell'utente e l'area funzionale richiesta (es. area supervisore).

Modello RBAC adottato

Il sistema OikoNaos adotta un modello Role-Based Access Control (RBAC) semplificato, coerente con l'implementazione attuale. Gli unici attori operativi che interagiscono con la piattaforma sono: Coinquilino e Supervisore di comunità

Il ruolo è associato all'utente come attributo applicativo e viene valutato dinamicamente durante l'elaborazione delle richieste. Non è prevista, nello stato attuale del sistema, un'area applicativa dedicata all'Azienda Coordinatrice.

Autorizzazione delle operazioni

L'autorizzazione è applicata a livello dei **controller** e dei **sottosistemi funzionali**, assicurando che ogni operazione sia consentita solo se:

- l'utente è autenticato;
- il ruolo è compatibile con l'operazione richiesta;
- lo stato del dominio applicativo lo consente (es. ticket aperto/modificabile).

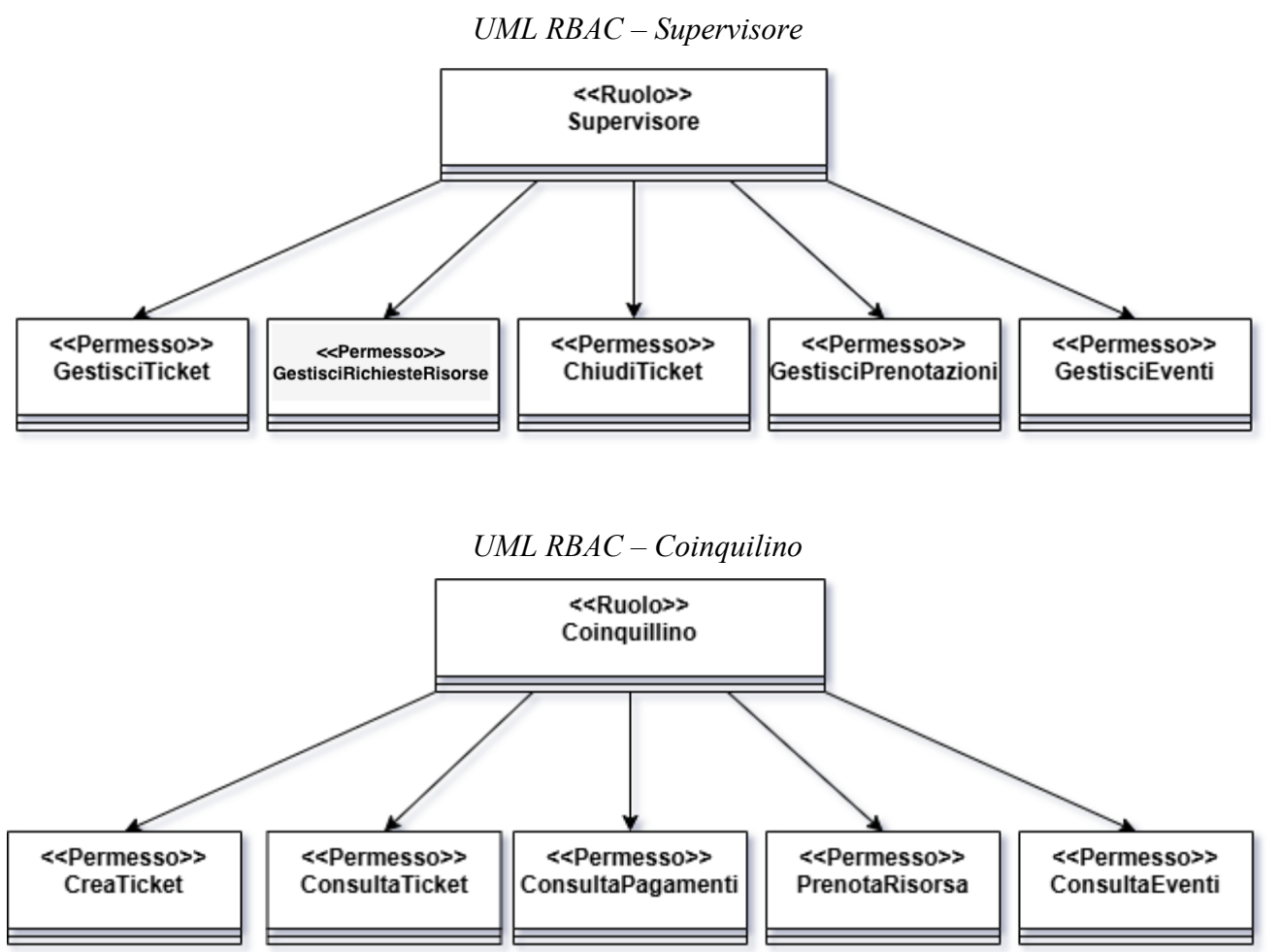
In particolare, l'accesso agli **allegati dei ticket** è controllato server-side: il download dei file è consentito esclusivamente agli utenti autorizzati, impedendo l'accesso diretto non mediato dal sistema.

Misure di sicurezza applicative

Oltre al controllo degli accessi, il sistema adotta ulteriori misure di sicurezza coerenti con l'implementazione:

- utilizzo di PreparedStatement per l'accesso al database, riducendo il rischio di SQL Injection;
- separazione dei privilegi tra applicazione e DBMS;
- validazione dei parametri di input lato server;
- protezione delle funzionalità sensibili tramite sessione e ruolo.

Le politiche avanzate di logging, audit e data retention non sono implementate a livello applicativo nello stato attuale del progetto e sono considerate responsabilità dell’infrastruttura o di future evoluzioni del sistema.



Il sistema OikoNaos adotta un modello di controllo degli accessi basato sui ruoli (RBAC). Gli unici attori che interagiscono operativamente con la piattaforma sono il Coinquilino e il Supervisore di comunità. La Tabella riassume, per ciascuna macro-area funzionale, le operazioni consentite ai due ruoli, in coerenza con i requisiti funzionali e con i casi d’uso definiti nel RAD.

Aerea/Operazioni Principali	Coinquilino	Supervisore
Gestione utenza e autenticazione	- Registrazione alla piattaforma tramite Codice Identificativo Speciale - Login / Logout - Recupero password - Visualizzazione del proprio profilo - Modifica dei propri dati personali (es. contatti, preferenze) - Modifica della propria password	- Login / Logout area supervisore - Visualizzazione del proprio profilo - Visualizzazione dell’elenco dei coinquilini della propria comunità (dati essenziali, es. nome, cognome, alloggio)
Gestione comunità e	Visualizzazione della comunità e	Visualizzazione struttura della

appartenenza alloggio	dell'alloggio associati al proprio account	comunità (alloggi, coinquilini associati) • Consultazione stato degli alloggi (occupato/libero, segnalazioni aperte)
Prenotazione spazi comuni	• Visualizzazione calendario spazi comuni disponibili • Creazione di una prenotazione per uno spazio/postazione • Modifica o cancellazione di una propria prenotazione entro i limiti previsti dal regolamento • Visualizzazione dello storico delle proprie prenotazioni	• Visualizzazione del calendario completo delle prenotazioni della comunità (tutti i coinquilini) • Cancellazione o modifica forzata di prenotazioni in casi eccezionali (es. manutenzione, eventi di comunità) • Blocco/sblocco di slot orari per esigenze di servizio
Ticket manutenzione di	• Creazione di un nuovo ticket di segnalazione (guasto, problema negli spazi comuni, ecc.) • Allegato di eventuali note e dettagli • Cancellazione del ticket finché è in stato "Aperto" • Visualizzazione elenco e stato dei propri ticket (Aperto, In lavorazione, Chiuso)	• Visualizzazione di tutti i ticket della comunità • Assegnazione/prioritizzazione dei ticket • Aggiornamento dello stato del ticket (Aperto → In lavorazione → Chiuso) • Inserimento di note di lavorazione e messaggi di risposta rivolti al coinquilino
Bacheca eventi e annunci	• Visualizzazione della bacheca eventi e degli annunci pubblicati per la propria comunità • Iscrizione/disiscrizione a un evento • Visualizzazione dell'elenco eventi a cui è iscritto	• Creazione di nuovi eventi per la comunità • Modifica/cancellazione di eventi esistenti • Pubblicazione di annunci informativi sulla bacheca • Visualizzazione delle iscrizioni ai singoli eventi (partecipanti)
Gestione risorse condivise	• Visualizzazione elenco risorse condivise disponibili nella comunità • Invio di richieste di utilizzo di una risorsa (se previsto dal modello dati) • Visualizzazione dello storico delle proprie richieste/assegnazioni	• Configurazione e aggiornamento delle risorse disponibili nella comunità (attive/non attive) • Approvazione o rifiuto delle richieste di utilizzo risorse, se previsto • Registrazione di eventuali violazioni/regole non rispettate
Tassa trimestrale e pagamenti	• Visualizzazione dell'importo della propria tassa trimestrale e dello stato (pagata / non pagata / in scadenza) • Pagamento on-line della tassa tramite il canale integrato • Visualizzazione storico dei propri pagamenti e delle ricevute	• Visualizzazione della situazione pagamenti dei coinquilini della propria comunità (chi ha pagato, chi è in ritardo) • Eventuale registrazione manuale di un pagamento ricevuto con modalità esterne al sistema (es. bonifico, contanti), se previsto
Mappa della struttura	• Visualizzazione della mappa interattiva della struttura (spazi comuni, alloggi, punti di interesse,	• Stessa visualizzazione della mappa, con eventuali informazioni aggiuntive utili alla gestione (es. spazi chiusi per

5.6. Global software control

Il controllo globale del software in OikoNaos segue il modello di esecuzione tipico delle **web application servlet-based**, basate su application server come **Apache Tomcat 10.1**. Ogni richiesta HTTP proveniente dal browser viene intercettata dal container e assegnata a un **thread** disponibile all'interno del pool di esecuzione, consentendo la gestione concorrente di più utenti in modo isolato ed efficiente.

Il flusso di controllo dell'applicazione è coerente con l'architettura **Model–View–Controller (MVC)**. Le interazioni dell'utente con l'interfaccia web (View) generano richieste HTTP verso il server; tali richieste vengono instradate da Tomcat al **controller servlet** appropriato, che interpreta il caso d'uso da eseguire. Il controller coordina quindi le operazioni applicative necessarie, interagendo con il **Model**, che incapsula le entità del dominio e i componenti di accesso ai dati (DAO). Al termine dell'elaborazione, il controller seleziona la vista JSP da restituire, che genera la risposta inviata al browser.

La **gestione della concorrenza** è affidata a una combinazione di meccanismi forniti dal container e dal database. A livello applicativo, ogni richiesta è gestita in modo indipendente da un thread dedicato, evitando interferenze dirette tra le richieste degli utenti. Prima di eseguire operazioni di modifica dei dati (ad esempio creazione di prenotazioni o aggiornamento dello stato dei ticket), i controller applicano **controlli e validazioni sullo stato corrente del dominio**, prevenendo operazioni non consentite o incoerenti. A livello di persistenza, il **DBMS** garantisce l'integrità dei dati tramite vincoli strutturali (chiavi primarie ed esterne) e meccanismi interni di sincronizzazione sulle singole operazioni di scrittura. Le operazioni di accesso ai dati sono eseguite in modalità **autocommit**, assicurando l'atomicità delle singole query, senza introdurre una gestione esplicita delle transazioni a livello applicativo.

La gestione degli errori avviene principalmente nei **controller**, che intercettano eccezioni e condizioni anomale durante l'elaborazione delle richieste. In caso di errore, il sistema interrompe l'operazione corrente e reindirizza l'utente verso viste di errore o messaggi informativi, evitando l'esposizione di dettagli interni o sensibili. Non è presente, nello stato attuale del sistema, un sottosistema applicativo dedicato al logging o all'audit delle operazioni.

Grazie al modello thread-per-request di Tomcat, alla separazione chiara tra View, Controller e Model e ai controlli applicativi sullo stato delle risorse, OikoNaos garantisce un flusso di controllo stabile e coerente anche in presenza di accessi concorrenti. Questo approccio consente al sistema di operare correttamente in contesti multiutente, tipici delle comunità di co-housing, mantenendo coerenza dei dati e affidabilità del servizio.

5.7. Boundary conditions

Le Boundary Conditions descrivono le condizioni iniziali, finali ed eccezionali che il sistema OikoNaos deve gestire per garantire un funzionamento corretto e coerente con la propria architettura. Esse

comprendono le precondizioni necessarie all'esecuzione delle operazioni, le postcondizioni garantite al termine dell'elaborazione e le modalità di gestione delle condizioni di errore a livello applicativo.

Precondizioni

Le principali precondizioni richieste dal sistema includono:

- la presenza di un **utente autenticato** per l'accesso alle funzionalità protette;
- la validità del **Codice Identificativo** durante la fase di registrazione;
- la correttezza e completezza dei **parametri** ricevuti dai controller;
- la compatibilità del **ruolo dell'utente** con l'operazione richiesta;
- la disponibilità logica delle risorse coinvolte (ad esempio, esistenza della prenotazione o del ticket da modificare).

Se una o più precondizioni non risultano soddisfatte, il controller interrompe l'elaborazione e restituisce una risposta informativa all'utente.

Postcondizioni

Al termine di un'operazione eseguita con successo, il sistema garantisce che:

- i dati siano stati **persistiti correttamente** nel database;
- lo stato del dominio applicativo risulti **coerente** con le regole di business;
- l'utente riceva una **conferma dell'operazione** tramite l'interfaccia web;
- le viste restituite riflettano in modo consistente l'aggiornamento effettuato.

Le operazioni di persistenza sono eseguite in modalità **autocommit**, garantendo l'atomicità delle singole operazioni di scrittura.

Gestione delle condizioni di errore

Durante l'elaborazione delle richieste, il sistema può incontrare condizioni di errore quali:

- credenziali errate o sessione non valida;
- tentativi di accesso a funzionalità non autorizzate;
- parametri non validi o mancanti;
- operazioni non consentite dallo stato corrente del dominio.

Tali condizioni vengono gestite a livello di **controller**, che intercettano le anomalie e reindirizzano l'utente verso **pagine di errore** o messaggi informativi, evitando l'esposizione di dettagli interni o sensibili dell'applicazione.

Condizioni eccezionali di sistema

Eventuali condizioni eccezionali di natura infrastrutturale, come l'indisponibilità del database o l'arresto dell'application server, non sono gestite direttamente dall'applicazione ma dipendono dall'ambiente di esecuzione. In tali casi, il sistema può risultare temporaneamente non disponibile e restituire messaggi di errore generici (es. *servizio non disponibile*), senza consentire la modifica del dominio applicativo fino al ripristino delle componenti esterne.

Start-up e shut-down del sistema

Le operazioni di avvio e arresto dell'applicazione seguono il ciclo di vita standard delle web application Java. L'avvio richiede la disponibilità del DBMS MySQL e dell'application server Apache Tomcat,

all'interno del quale la WebApp viene deployata. In caso di indisponibilità delle componenti necessarie durante l'avvio, l'applicazione non risulta accessibile. Le procedure di shut-down sono demandate al container e al sistema operativo, senza gestione applicativa esplicita delle sessioni attive.

Operazioni amministrative esterne

Alcune operazioni amministrative non rientrano nelle funzionalità esposte dalla WebApp. In particolare, la gestione diretta degli account con privilegi elevati (es. creazione o rimozione di supervisor) è demandata a **procedure esterne** all'applicazione, come l'intervento manuale sul database da parte di un operatore autorizzato. Questa scelta progettuale riduce la superficie di attacco e limita l'esposizione di funzionalità critiche.

6. Subsystem Service Glossary

Questa sezione raccoglie e descrive in modo sintetico tutti i servizi offerti dai sottosistemi logici di OikoNaos.

Ogni sottosistema del livello *Controller* espone un insieme di operazioni che implementano i casi d'uso definiti nel RAD, mentre il livello *Model* fornisce servizi di persistenza e gestione delle entità del dominio.

Le seguenti tabelle riassumono i servizi principali, raggruppati per sottosistema.

Gestione Autenticazione

Servizio	Descrizione
Autenticazione	Verifica delle credenziali dell'utente e creazione di una sessione autenticata.
Logout	Terminazione della sessione utente e invalidazione dei dati di sessione.
Recupero password	Gestione del flusso di reset password tramite token temporaneo.
Validazione codice identificativo	Verifica della validità del codice fornito in fase di prima registrazione.
Identificazione ruolo	Associazione dell'utente autenticato al ruolo applicativo (coinquilino o supervisore).

Gestione Profilo Utente

Servizio	Descrizione
Visualizza profilo	Recupero dei dati anagrafici e di contatto dell'utente autenticato.
Modifica informazioni	Aggiornamento dei dati personali dell'utente (es. contatti).
Modifica credenziali	Aggiornamento della password nel rispetto dei requisiti di sicurezza.

Gestione Prenotazione

Servizio	Descrizione
Creazione prenotazione	Inserimento di una prenotazione per uno spazio/postazione in una specifica fascia temporale.
Modifica prenotazione	Aggiornamento dei dati di una prenotazione esistente, se consentito.

Cancellazione prenotazione	Eliminazione di una prenotazione da parte dell'utente o del supervisore.
Verifica disponibilità	Controllo della disponibilità logica dello spazio richiesto.
Visualizza prenotazioni	Consultazione delle prenotazioni attive e dello storico.

Gestione Ticket di Manutenzione

Servizio	Descrizione
Creazione ticket	Inserimento di una segnalazione di manutenzione con descrizione.
Allegazione file	Associazione di file (immagini/documenti) a un ticket.
Aggiornamento ticket	Aggiornamento dello stato e inserimento di note di lavorazione (supervisore).
Chiusura ticket	Chiusura del ticket e registrazione dello stato finale.
Visualizza ticket	Consultazione dei ticket e del loro stato.
Storici aggiornamenti	Visualizzazione delle modifiche e degli aggiornamenti associati al ticket.

Gestione Eventi

Servizio	Descrizione
Creazione evento	Creazione di un evento da parte del supervisore.
Pubblicazione evento	Inserimento dell'evento nella bacheca della comunità.
Iscrizione evento	Iscrizione o disiscrizione dell'utente a un evento.
Visualizza eventi	Consultazione degli eventi disponibili e passati.
Gestione posti disponibili	Aggiornamento del numero di posti disponibili per un evento.

Gestione Risorse Condivise

Servizio	Descrizione
Richiesta risorsa	Invio di una richiesta di utilizzo di una risorsa condivisa.
Gestione disponibilità	Aggiornamento dello stato di una risorsa (attiva/non attiva).
Visualizza risorse	Consultazione dell'elenco delle risorse disponibili.
Gestione richieste risorsa	Valutazione e gestione delle richieste da parte del supervisore.

Gestione Pagamenti

Servizio	Descrizione
Visualizzazione spese	Consultazione dell'importo della tassa trimestrale.
Registrazione pagamento	Registrazione dello stato di pagamento della tassa.
Verifica stato pagamento	Consultazione dello stato della tassa (pagata / non pagata).
Consultazione storico	Visualizzazione dello storico dei pagamenti e delle ricevute.

Sottosistema Model – Persistenza Dati

Servizio	Descrizione
CRUD entità	Operazioni di creazione, lettura, aggiornamento ed eliminazione delle entità del dominio.

Recupero dati	Esecuzione di query applicative a supporto dei casi d'uso.
Gestione allegati	Salvataggio e recupero dei file associati ai ticket.
Vincoli di integrità	Applicazione dei vincoli relazionali definiti nello schema del database.